

תכנות תחרותי

הרצאה 1



מה בתוכנית?

- איך נראה קוד cpp
- STL
- CSES, codeforces, אתר הקורס
- Greedy problems
- Dynamic Programming

cpp code

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c)

signed main() {
    int n; cin >> n;
    vector<int> a(n);
    for(int i = 0; i < n; i++) cin >> a[i];

    sort( first: a.begin(), last: a.end());

    int x = a[4];
    x += 5;
    cout << x << "\nHello world!" << endl;

    return 0;
}
```

```
#include <bits/stdc++.h>
```

שורה אחרי שורה

הספריה הזו מכילה את רוב מה שצריך על מנת לכתוב קוד.

הספריה כוללת מבני נתונים, פעולות מתמטיקה, פעולות קלט ופלט ועוד!

```
#include <bits/stdc++.h>
using namespace std;
```

שורה אחרי שורה

על מנת שנוכל להשתמש באובייקטים מהספריה בלי
להזכיר בקוד שהאובייקטים האלו לקוחים מתוך
הספריה

✗ `std::vector<int>`

✓ `vector<int>`

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
```

שורה אחרי שורה

במקום int שהוא 32 ביט נשתמש ב long long
שהוא 64 ביט, יחסוך לנו התעסקות עם overflow

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)
```

שורה אחרי שורה

הגדרת macro עבור לולאה

בתכנות תחרותי נרצה לכתוב קוד כמה שיותר מהר,
בשביל זה נשתמש בשלל macros כדי לקצר את
התהליך

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)

signed main() {
```

שורה אחרי שורה

למה signed main? מה קרה ל-int main?

!!! #define int long long

זהו פשוט שם אחר ל-int !

נדרשת פתיחת סוגריים, {, לבאים מפייתון


```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)

signed main() {
    int n; cin >> n;
```

שורה אחרי שורה

הצהרת המשתנה n וקליטה שלו

אנחנו משתמשים ב-cin אשר לקוח מהספרייה על מנת
לבצע קלט

; אסור לשכוח בסוף שורה

שורה אחרי שורה

הצהרה על וקטור מאורך n

בינתיים זהו מערך עבורנו, עוד מעט נבין אותו לעומק

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)

signed main() {
    int n; cin >> n;
    vector<int> a(n);
```

שורה אחרי שורה

מילוי הוקטור בקלט על ידי לולאה, ככה נראת לולאת
for ב-cpp

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)

signed main() {
    int n; cin >> n;
    vector<int> a(n);

    for (int i=0; i < n; i++) cin >> a[i];
```

שורה אחרי שורה

הצהרה על משתני counter,pos

counter - המונה שלנו

pos - יציין לנו את המיקום שלנו במערך

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)

signed main() {
    int n; cin >> n;
    vector<int> a(n);

    for (int i=0; i < n; i++) cin >> a[i];

    int counter = 0;
    int pos = 0;
```

שורה אחרי שורה

ככה נראית לולאת while ב-cpp

הלולאה תרוץ עד שהערך ב-pos יהיה גדול או שווה
לערך ב-n

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)

signed main() {
    int n; cin >> n;
    vector<int> a(n);

    for (int i=0; i < n; i++) cin >> a[i];

    int counter = 0;
    int pos = 0;
    while (pos < n) {
```

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)

signed main() {
    int n; cin >> n;
    vector<int> a(n);

    for (int i=0; i < n; i++) cin >> a[i];

    int counter = 0;
    int pos = 0;
    while (pos < n) {
        if (a[pos] >= 10 || a[pos] == 5) counter += 1;
        else pos++;
    }
}
```

שורה אחרי שורה

תנאי if,else ב-cpp

|| זה תנאי or, && תנאי and

pos++ מגדיל את pos ב-1

שורה אחרי שורה

דרך מהירה ונוחה לבצע פעולות אריתמטיות במקום

לכתוב $\text{counter} = \text{counter} * 2$

(במקום * יכול להיות גם +, -, ...)

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)

signed main() {
    int n; cin >> n;
    vector<int> a(n);

    for (int i=0; i < n; i++) cin >> a[i];

    int counter = 0;
    int pos = 0;
    while (pos < n) {
        if (a[pos] >= 10 || a[pos] == 5) counter += 1;
        else pos++;
    }

    counter *= 2;
```

שורה אחרי שורה

הדפסה של x, שוב פעולה שלקוחה מתוך הספרייה

אנחנו נשתמש ב-cout עבור הדפסות

\n? endl?

יש הבדל בכלל?

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)

signed main() {
    int n; cin >> n;
    vector<int> a(n);

    for (int i=0; i < n; i++) cin >> a[i];

    int counter = 0;
    int pos = 0;
    while (pos < n) {
        if (a[pos] >= 10 || a[pos] == 5) counter += 1;
        else pos++;
    }

    counter *= 2;

    cout << "Value of counter is: "
    << counter << "\nGoodbye" << endl;
```


שורה אחרי שורה

חזרה

סגירת סוגריים, }

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)

signed main() {
    int n; cin >> n;
    vector<int> a(n);

    for (int i=0; i < n; i++) cin >> a[i];

    int counter = 0;
    int pos = 0;
    while (pos < n) {
        if (a[pos] >= 10 || a[pos] == 5) counter += 1;
        else pos++;
    }

    counter *= 2;

    cout << "Value of counter is: "
    << counter << "\nGoodbye" << endl;

    return 0;
}
```

כמה מבני נתונים חשובים מהספרייה שלנו

- Pair & Tuple
- Vector
- String
- Set & Multiset
- Map & Unordered Map
- Priority queue (Heap)
- Queue & Stack
- deque

C++ reference

C++11, C++14, C++17, C++20, C++23, C++26, C++29 | Compiler support C++11, C++14, C++17, C++20, C++23, C++26, C++29

Language

Preprocessor – Comments
ASCII chart
Basic concepts
Keywords
Names (lookup)
Types (fundamental types)
The main function
Modules (C++20)
Contracts (C++26)
Expressions
Value categories
Evaluation order
Operators (precedence)
Conversions – Literals
Constant expressions
Statements
if – switch
for – range- for (C++11)
while – do – while
Declarations – Initialization
Functions – Overloading
Coroutines (C++20)
Classes (unions)
Templates – Exceptions
Freestanding implementations

Standard library (headers)

Named requirements

Language support library

Program utilities
Signals – Non-local jumps
Basic memory management
Variadic functions
source_location (C++20)
Comparison utilities (C++20)
Type support – type_info
numeric_limits – exception
initializer_list (C++11)
Coroutine support (C++20)
Contract support (C++26)

Concepts library (C++20)

Diagnostics library

Assertions – System error (C++11)
Exception types – Error numbers
basic_stacktrace (C++23)
Debugging support (C++26)

Memory management library

Allocators – Smart pointers
Memory resources (C++17)

Metaprogramming library (C++11)

Type traits – ratio
integer_sequence (C++14)
Reflection (C++26)

General utilities library

Function objects – hash (C++11)
Swap – Type operations (C++11)
Integer comparison (C++20)
pair – tuple (C++11)
optional (C++17)
expected (C++23)
variant (C++17) – any (C++17)
bitset – Bit manipulation (C++20)

Containers library

vector – deque – array (C++11)
list – forward_list (C++11)
inplace_vector (C++26)
hive (C++26)
map – multimap – set – multiset
unordered_map (C++11)
unordered_multimap (C++11)
unordered_set (C++11)
unordered_multiset (C++11)
Container adaptors
span (C++20) – mdspan (C++23)

Iterators library

Ranges library (C++20)

Range factories – Range adaptors
generator (C++23)

Algorithms library

Numeric algorithms
Execution policies (C++17)
Constrained algorithms (C++20)

Strings library

basic_string – char_traits
basic_string_view (C++17)

Text processing library

Primitive numeric conversions (C++17)
Formatting (C++20) – Localization
text_encoding (C++26)
Regular expressions (C++11)
basic_regex – Algorithms
Default regular expression grammar
Null-terminated sequence utilities:
byte – multibyte – wide

Numerics library

Common math functions
Mathematical special functions (C++17)
Mathematical constants (C++20)
Basic linear algebra algorithms (C++26)
Data-parallel types (SIMD) (C++26)
Pseudo-random number generation
Floating-point environment (C++11)
complex – valarray

Date and time library

Calendar (C++20) – Time zone (C++20)

Input/output library

Print functions (C++23)
Stream-based I/O – I/O manipulators
basic_istream – basic_ostream
Synchronized output (C++20)
File systems (C++17)

Concurrency support library (C++11)

thread – jthread (C++20)
atomic – atomic_flag
atomic_ref (C++20) – memory_order
Mutual exclusion – Condition variables
Futures – Semaphores (C++20)
latch (C++20) – barrier (C++20)
Safe Reclamation (C++26)

Execution support library (C++26)

Feature test macros (C++20)

Language – Standard library – Headers

```
int x = 3, y=4;  
pair<int,int> pair = { [&]x, [&]y};  
int f = pair.first + pair.second;
```

- Pair - an Ordered pair of 2 elements - like coordinates

```
tuple<int, int, float> tuple = {1, 2, 3.33333};  
int f = get<0>([&]tuple);
```

- Tuple - an Ordered group for a number of elements

Vector

```
vector<int> v1 = {1,2,3,3,4,5};  
v1.push_back(x:1000);  
for (auto i:long long : v1) {  
    cout<<i<<" ";  
}  
cout<<endl;  
vii v2 = {{x:1,y:2},{x:3,y:4}}; //vector<pair<int,int>>  
for (auto [a:long long , b:long long] : v2) {  
    cout<<a<<" "<<b<<"\n";  
}  
auto it:ptrdiff_t = lower_bound(first:v1.begin(), last:v1.end(), val:3) - v1.begin();  
cout << it << endl;  
sort(first:v1.begin(), last:v1.end());
```

1 2 3 3 4 5 1000

1 2

3 4

2

Vector is like an array that you can make larger in runtime.

- Line 2 - add the element 1000 to the end of the vector
- Line 3 - iterating by value
- Line 8 - iterating over all pairs in v2
- Line 11 - returns the index to the first occurrence of val = 3 in a sorted array (binary search).
- Sort - sorts

Sorting

```
vii v = {{x:1, y:313}, {x:2, y:0}, {x:3, y:100}};
sort(first:all(v), sorter);
for (auto [a:long long, b:long long] : v) {
    cout<<a<<" "<<b<<"\n";
}
cout<<"\n";
sort(first:all(v));
for (auto [a:long long, b:long long] : v) {
    cout<<a<<" "<<b<<"\n";
}
```

```
2 0
3 100
1 313

1 313
2 0
3 100
```

```
bool sorter(pair<int,int> a, pair<int,int> b) {
    if (a.second < b.second) return true;
    if (a.first < b.first) return true;
    return false;
}

bool sorter(pair<int,int> a, pair<int,int> b) { //default pair sorter
    if (a.first < b.first) return true;
    if (a.second < b.second) return true;
    return false;
}
```

You can easily sort a vector in the most efficient way (optimized $n\log n$) using the `sort()` function. If you want custom sorting (for example if you sort pairs and want them to be sorted by the second and then the first), you can make a sort function and put it in the sort

String

```
string s = "competitive programming";  
if (s[0] == 'c')  
    cout << s << '\n';  
cout << s.substr(pos:12) << '\n';
```

```
competitive programming  
programming
```

- Access - like arrays
- Substr - returns a suffix from pos to end
- There are a lot more functions - can see in cppreference

Set & Multiset

```
set<int> set1;
set1.insert(x:2);
set1.insert(x:1);
cout << *set1.begin() << '\n'; // prints minimum
auto it = set1.lower_bound(x:2);
if (it != set1.end()) { // set1.end() returns an iterator to the position just after the last element
    cout << *it << '\n'; // prints the value that is the largest while being <=2
}
set1.erase(x:2); // remove if contained
```

- The idea is an avl tree (with values being the key too) so sets and multisets are sorted. Set cannot include a single element multiple times, multiset can.
- If you want set with multiple instances of the same element you can do `set<pair<element,int>>`, and give each element a different number, now they don't count as the same.

Set & Multiset

```
multiset<int> ms;  
ms.insert(x:5);  
ms.insert(x:2);  
ms.insert(x:5); // Duplicates are allowed  
  
auto it = ms.find(x:5); // Remove only one copy of 5  
if (it != ms.end())  
    ms.erase(it);  
ms.erase(x:5); // Remove only all copies of 5
```


Map & Unordered Map

```
map<char,int> mp;  
mp['1'] = 1;  
mp['2'] = 3;  
for (auto [key:const char, value:long long] : mp)  
    cout << key << ":" << value << '\n';
```

- This is an AVL tree that has keys that may differ from their corresponding value.
- This data structure is useful where $n \leq 2 \cdot 10^5$, but $x \leq 10^9$.
- Also useful for mapping from string to int or similar.

Map & Unordered Map

```
unordered_map<char,int> ump;  
ump['1'] = 1;  
ump['2'] = 3;  
for (auto [key:const char, value:long long] : ump)  
    cout << key << ":" << value << '\n';
```

- This is NOT an AVL tree, but a hash map that has keys that may differ from their corresponding value.
- Usually we just use map instead

Priority queue (Heap)

```
priority_queue<int> q; //maximum heap
q.push(x:1);
int a = q.top();
q.pop();
priority_queue<int, vector<int>, greater<int>> q2; //minimum heap
```

- Priority queue is a maximum heap, or if you want you can change it to be a minimum heap.
- Acts as a sorted queue

Queue & Stack

```
queue<int> q;  
q.push(x:1);  
int a = q.front();  
q.pop();
```

```
stack<int> s;  
s.push(x:1);  
int a = s.top();  
s.pop();
```

Just a normal queue and a normal stack

deque

Its like a queue and stack together (can choose if to push to front or back), but most of the time you just use queue or stack.

```
deque<int> d;  
d.push_back(x:0);  
d.push_front(x:0);  
int a = d.front();  
int b = d.back();  
d.pop_front();  
d.pop_back();
```

קוד חשוב ונוח

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define rep(a,b,c) for(int a=b;a<c;a++)
#define all(x) x.begin(),x.end()
typedef pair<int,int> pii;
typedef vector<int> vi;

void solve() {
    int n; cin >> n;
}

signed main() {
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

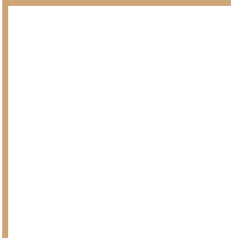


Competitive Programming At University of Haifa

Welcome to the Competitive Programming At University of Haifa website!

This website is designed to help students learn and practice competitive programming.

זמן לתרגל!



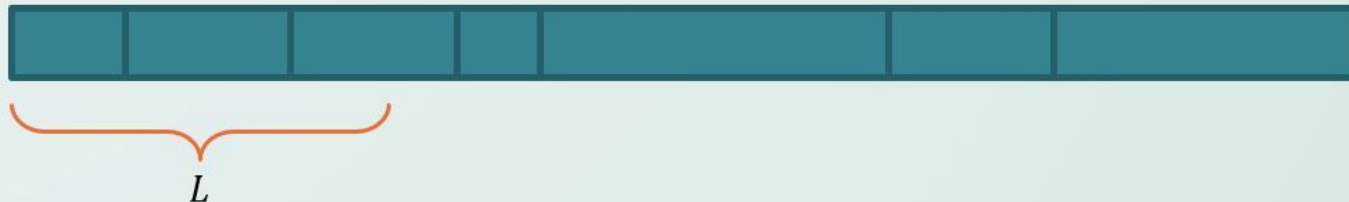
greedy

הכי פשוט שיש



בעיית הנגר

- נגר מקבל קורת עץ לחיתוך באורך כולל S .
- על הקורה מסומנים n מיקומים אפשריים לחיתוך $x_1, x_2, \dots, x_n$, שרק בהם יכול הנגר לחתוך את הקורה.
- מזמין העבודה זקוק לקורות באורך לכל היותר L . בפרט, הוא רוצה שלאחר החיתוך, כל חלקי הקורות יהיו לא יותר ארוכות מ- L .
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, שיבחר את מקומות החיתוך, כך שמספר הקורות באורך לכל היותר L יהיה מינימלי. הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.

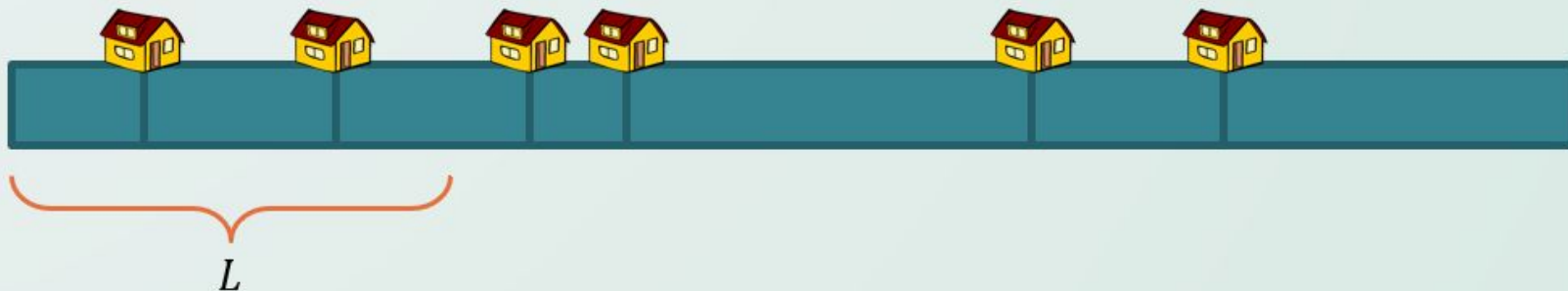


מהי בעיה חמדנית?

A greedy algorithm solves optimization problems by making the locally optimal choice at each step, hoping to find the global optimum. It is efficient but doesn't always guarantee the best overall solution. Key properties include the greedy choice property and optimal substructure.

סיפור אחר, אותה הבעיה

- ישנו מסלול באורך S ועליו מסומנות n נקודות עצירה שונות $x_1, x_2, \dots, x_n$.
- אתם יודעים כי בכל יום תוכלו ללכת מרחק של לכל היותר L . עליכם כעת לבחור את מקומות הלינה, מבין n נקודות העצירה השונות.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, שיבחר את מקומות העצירה, כך שתסיימו את המסלול במספר ימים מינימלי. הוכיחו את נכונותו וסיבוכיותו של האלגוריתם שלכם.



איך פותרים שאלה חמדנית?

- רוב הבעיות אינן חמדניות ופתרון חמדני לא יעבוד עבורן
- בשביל לזהות שאלה עם אלגוריתם חמדני צריך לזהות תכונה עקרונית של הבעיה, תובנה כלשהי שמצדיקה את הבחירה החמדנית בכל שלב
- במילים אחרות: הפתרון מגיע מהבנה עמוקה יותר של השאלה

אז איך משתפרים?

גרידי הוא לא נושא עם הרבה חומר תאורטי, התקמדות בנושא מגיעה מתרגול של כמה שיותר שאלות כדי לזהות יותר בקלות מתי שאלה היא חמדנית

Taxi

- ישנן n קבוצות של חברים בגודל 1-4 חברים שמחכות ביחד בתחנת המוניות
- כל מונית יכולה לקחת עד 4 אנשים
- כל קבוצת חברים רוצה לנסוע באותה מונית אבל מונית אחת יכולה לקחת כמה קבוצות
- מה המספר המינימלי של מוניות הנדרש?
- סיבוכיות זמן לינארית $O(n)$

פתרון

- קבוצות חברים מגודל 4 חייבות לקחת מונית בעצמן
- קבוצות חברים מגודל 3 יכולות לקחת איתן במונית קבוצות חברים מגודל 1 ואין סיבה שהן לא יעשו זאת
- קבוצות חברים מגודל 2 תמיד יעדיפו לקחת איתן במונית קבוצה נוספת מגודל 2 על פני שתי קבוצות מגודל 1!

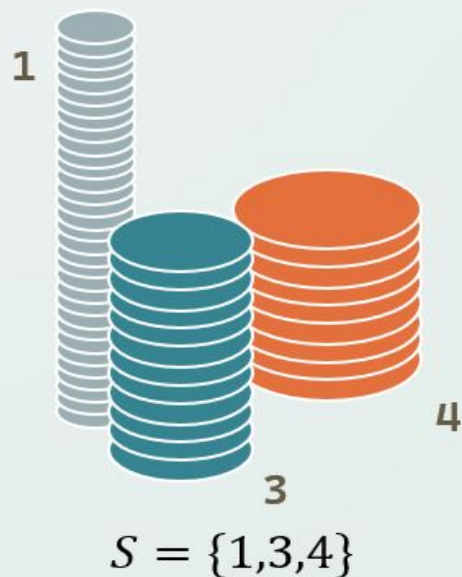
מכפלה של שלושה מספרים

- נתון מספר $n < 10^9$
- המטרה היא לקבוע האם קיימים שלושה מספרים שונים a, b, c כך ש $a * b * c = n$
- אם קיימים שלושה כאלה למצוא אותם (אם יש כמה אופציות אפשר להחזיר כל אחת)
- שימו לב לסיבוכיות הזמן שיש לנו, לינארי לא מהיר מספיק!

פתרון

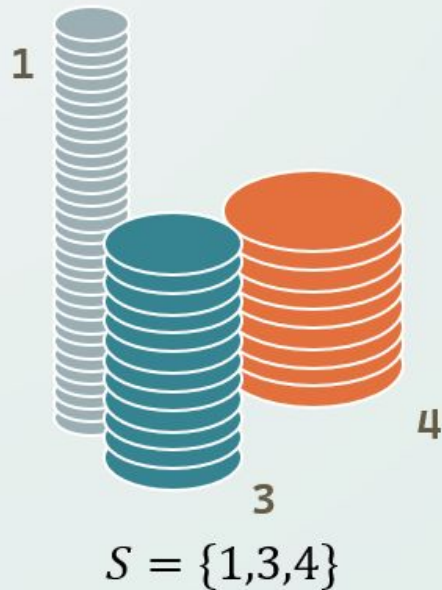
- אם קיים פתרון אז בהכרח קיים פתרון שבו a הוא המחלק הקטן ביותר של n . המחלק הזה בהכרח קטן שווה לשורש שלישי של n , נעבור מספר מספר ונחפש אותו.
- נשאר לנו לבדוק האם n/a ניתן לחלוקה על ידי שני מספרים שונים שהם גם שונים מ- a .
אם כן הקטן מביניהם צריך להיות קטן משורש n/a , נעבור מספר מספר ונחפש אותו.

הגדרה לבעיית המטבעות



- נתונה קבוצה של k ערכי מטבעות שלמים שונים $S = \{c_1, c_2, \dots, c_k\}$
- בנוסף נתון סכום מטרה n .
- נרצה להרכיב את הסכום n בעזרת מטבעות מתוך הקבוצה S .
- מהו מספר המטבעות המינימלי שנצטרך?
- באילו מטבעות נשתמש כדי לייצג את הסכום בצורה מינימלית?
- האם בכלל אפשר לייצג את הסכום n בעזרת המטבעות?

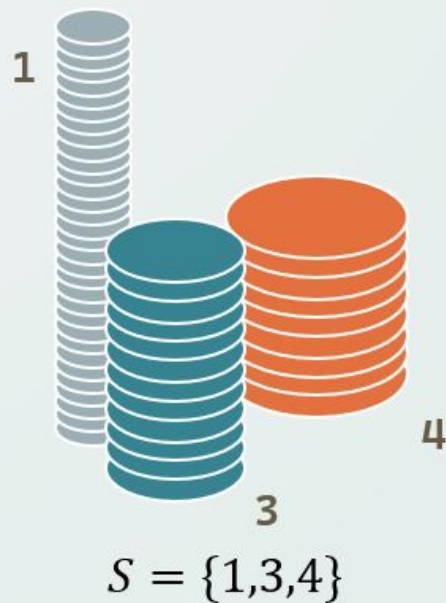
הפתרון החמדני



- פתרון אפשרי הינו הפתרון החמדני הבא:
- ניקח כל פעם את המטבע הכי גדול שהוספתו אינה חורגת מהסכום
- האם הפתרון יעבוד? כיצד נוכל לדעת אם לא ניתן להרכיב את הסכום n ?

הפתרון החמדני

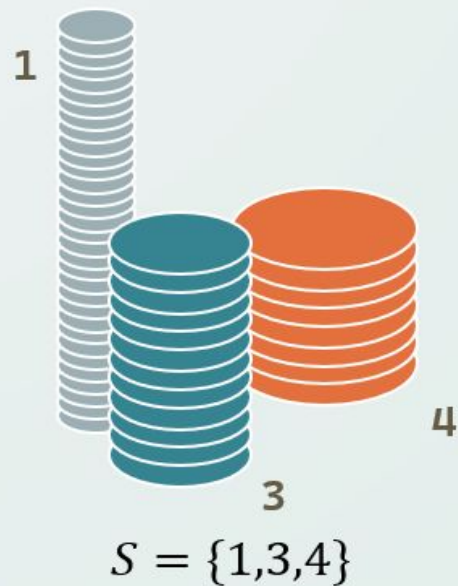
- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו- $n = 6$:



נותר לסכום: 6

הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו- $n = 6$:



נותר לסכום: 2



הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו- $n = 6$:

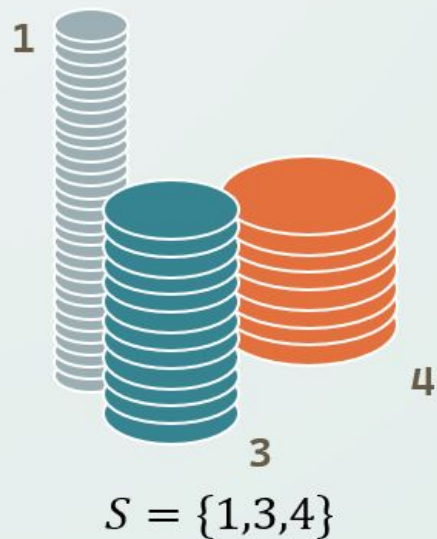


נוטר לסכום: 1

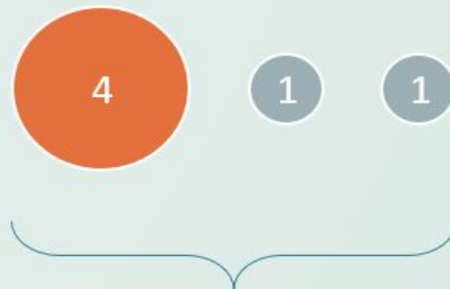


הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו- $n = 6$:



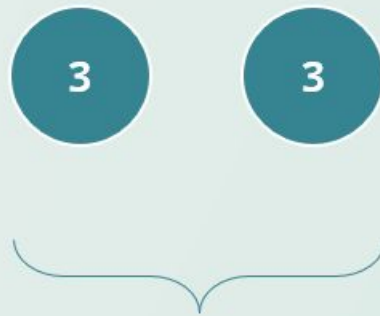
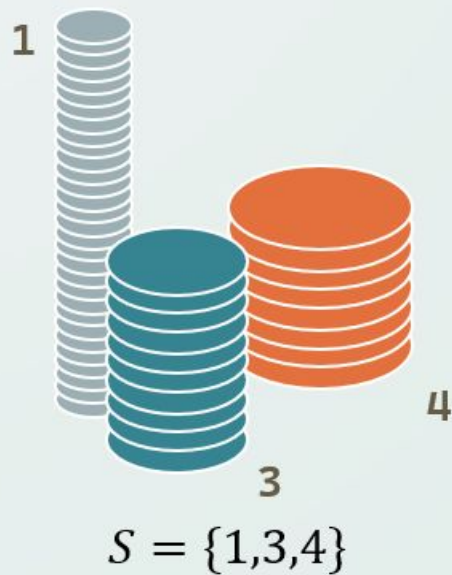
נותר לסכום: 0



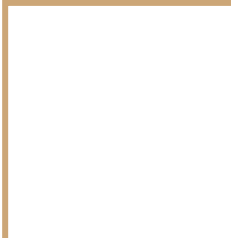
הפתרון החמדני נתן סכום
בעזרת 3 מטבעות

הפתרון החמדני

- נביא דוגמה לפתרון החמדני בעבור $S = \{1,3,4\}$ ו- $n = 6$:



ישנו פתרון טוב יותר בעזרת 2
מטבעות בלבד!



DP

תכנון דינאמי



מה זה בכלל תכנון דינמי?

הרעיון המרכזי מאחורי תכנות דינמי הוא מאוד פשוט: נשמור את הפתרונות עבור תת-בעיות קטנות, ונשתמש בהם כדי לפתור בעיות גדולות יותר – ובכך נחסוך חישובים מיותרים ונשפר משמעותית את זמן הריצה

עובדה לא כל כך מעניינת: התכנון הדינאמי הומצא על ידי יהודי ולשם dynamic programming אין כמעט משמעות זה בעיקר פשוט שם לא מאיים.



Richard Ernest Bellman

סדרת פיבונאצ'י

$$F_n = F_{n-1} + F_{n-2}$$

$$F(0) = 1$$

$$F(1) = 1$$

כאשר:

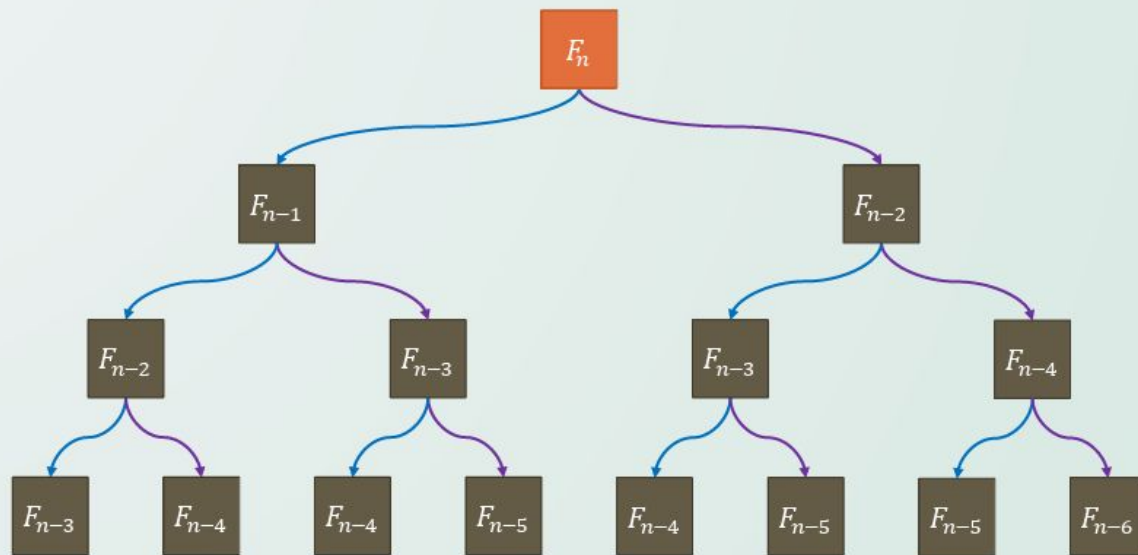
נפתור באופן נאיבי:

נרצה לחשב את האיבר ה-n בסדרה, ונכתוב קוד הפותר רקורסיבית באופן נאיבי את הבעיה:

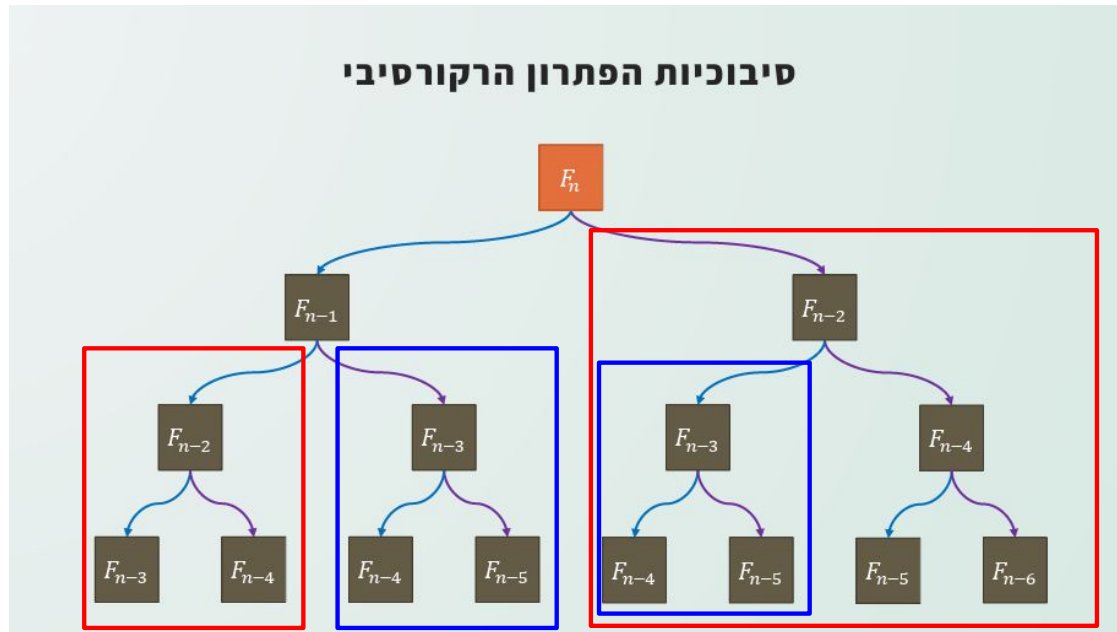
```
int fib(int n) {  
    if (n<2) return 1;  
    return fib(n-1)+fib(n-2);  
}
```

עץ הקריאות הרקורסיבי יראה כך:

סיבוכיות הפתרון הרקורסיבי



נשים לב כי נבצע חישובים של אותו האיבר מספר פעמים ללא סיבה



סיבוכיות נוראית: $O(n^2)$

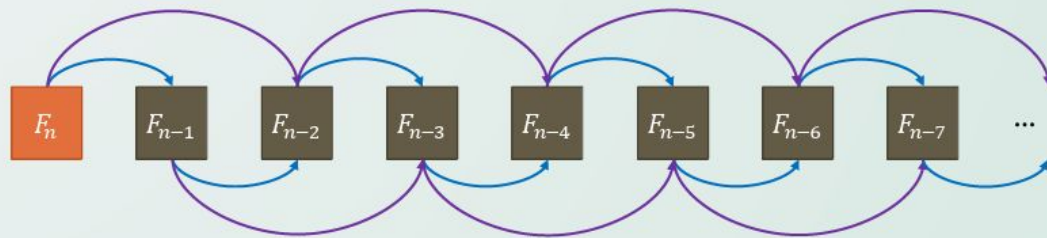


נשנה את הקוד כך שנשמור במערך חישובים שכבר בוצעו

```
vector<int> fib_mem(n:1e5, value:-1);  
int fib(int n) {  
    if (fib_mem[n] != -1) return fib_mem[n];  
    if (n<2) return n;  
    return fib_mem[n] = fib_mem[n-2]+fib_mem[n-1];  
}
```

הקריאות של הקוד החדש יראו כך:

מרקורסיה לאיטרציה



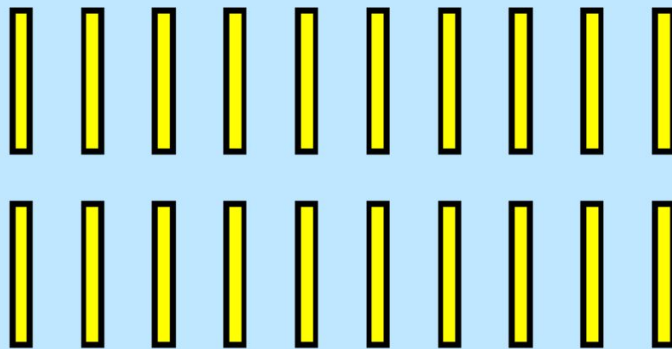
סיבוכיות מעולה: $O(n)$



דוגמא נוספת - 20 גפרורים

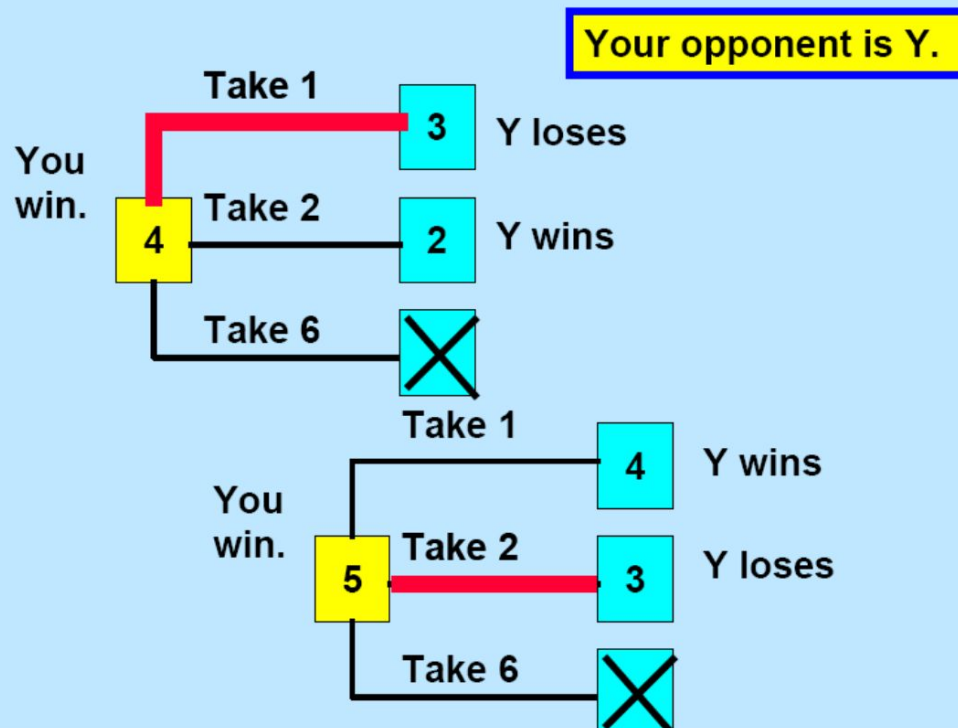
במשחק 20 גפרורים נרצה להחזיר מי מהשחקנים מנצח בסיבוכיות אופטימלית, נגדיר את המשחק באופן הבא:

- Choose a person to go first.
- Alternate in eliminating 1 or 2 or 6 matches.
- The person who takes the last match wins.



עץ החלטות עם מספר גפרורים של 4 או 5

Decisions with 4 or 5 matches left.



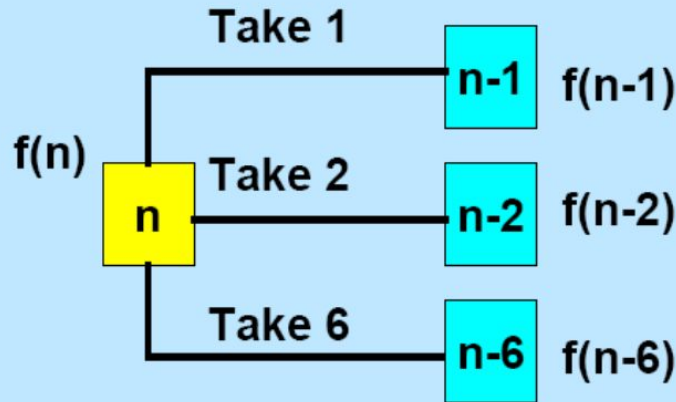
עץ החלטות עם מספר גפרורים של 6 או 7

Decisions with 6 or 7 matches left.



נפתור בעזרת תכנון דינמי:

Dynamic Programming Recursion



The Dynamic Programming
Recursion

$f(n) = W$ if $f(n-1) = L$
or $f(n-2) = L$
or $f(n-6) = L$;

$f(n) = L$ if $f(n-1) = f(n-2) = f(n-6) = W$.

איך הטבלה תראה עבור 50 גפרורים:

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

קוד

יש בעיה עם הקוד הזה?
מקרי קצה!!! מה אם $n=2$?

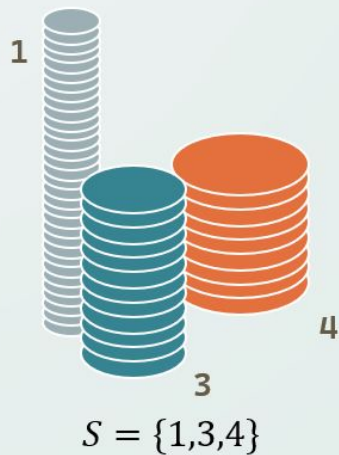
```
1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define rep(a,b,c) for(int a=b;a<c;a++)
5 #define per(a,b,c) for(int a=c-1;a>=b;a--)
6 typedef pair<int,int> pii;
7 typedef vector<int> vi;
8
9 void solve() {
10     int n; cin >> n;
11     vi dp(n+1);
12     dp[1] = 1, dp[2] = 1, dp[3] = 0, dp[4] = 1, dp[5] = 1, dp[6] = 1;
13
14     rep(i,7,n+1) {
15         if (dp[i-1] == 0 || dp[i-2] == 0 || dp[i-6] == 0)
16             dp[i] = 1;
17         else
18             dp[i] = 0;
19     }
20
21     (dp[n] == 1) ? cout << "win" << endl : cout << "lose" << endl;
22
23     return;
24 }
25
26 signed main() {
27     int t = 1;
28     //cin >> t;
29     while(t--) solve();
30     return 0;
31 }
```

סיבוכיות מעולה: (מספר הגפרורים) O



נחזור לשאלת המטבעות: פתור בסיבוכיות: $O(kn)$

הגדרה לבעיית המטבעות



- נתונה קבוצה של k ערכי מטבעות שלמים שונים $S = \{c_1, c_2, \dots, c_k\}$
- בנוסף נתון סכום מטרה n .
- נרצה להרכיב את הסכום n בעזרת מטבעות מתוך הקבוצה S .
- מהו מספר המטבעות המינימלי שנצטרך?
- באילו מטבעות נשתמש כדי לייצג את הסכום בצורה מינימלית?
- האם בכלל אפשר לייצג את הסכום n בעזרת המטבעות?

נפתור בעזרת DP:

```
using vi = vector<int>;
const int oo = 1e18;

signed main(){
    cin.tie(0)->sync_with_stdio(false), cout.tie(0);
    int n, x, c; cin >> n >> x;
    vi dp(n+1, oo); dp[0]=0;
    rep(i, 0, n){
        cin >> c;
        rep(j, 0, x-c+1) if(dp[j] != oo) dp[j+c] = min(dp[j+c], dp[j]+1);
    } cout << (dp[x] == oo ? -1 : dp[x]);
}
```

נשים לב שהפתרון האופטימלי עבור x בטוח יהיה הפתרון האופטימלי של x -coin מסוים, ועוד 1 כי נוסיף את המטבע הזה.

לכן נעבור בלולאה על המספרים מ 1 עד n , וכל אחד נעדכן להיות האופטימלי מבין הקודמים אליו בהפרש מטבע אחד.